

Program 1: Simple Signal Handler — SIGINT:

```
GNU nano 8.7.1 signal1.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    printf("\nSIGINT received!\n");
}

int main()
{
    signal(SIGINT, handle_signal);

    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}
```

```
compilation terminated.
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ nano signal1.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ gcc signal1.c -o signal1
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ ./signal1
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^XProgram is running...
Program is running...
Program is running...
^C
SIGINT received!
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^Z
[1]+  Stopped                  ./signal1
```

Program 2: SIGINT Without a Handler:

```
GNU nano 8.7.1 signal2.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}
```

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ nano signal2.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ gcc signal2.c -o signal2
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ ./signal2
Program is running...
Program is running...
Program is running...
Program is running...
^C

```

Program 3: SIGCHLD - Child Process Signal:

```

GNU nano 8.7.1 signal3.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handle_sigchld(int sig)
{
    printf("SIGCHLD received: Child has terminated.\n");
}
int main()
{
    signal(SIGCHLD, handle_sigchld);
    pid_t pid = fork();
    if (pid == 0)
    {
        // Child
        printf("Child is running...\n");
        sleep(3);
        printf("Child is terminating...\n");
    }
    else
    {
        // Parent
        printf("Parent is waiting...\n");
        wait(NULL);
        printf("Parent finished.\n");
    }
    return 0;
}

```

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ nano signal3.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ gcc signal3.c -o signal3
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ ./signal3
Parent is waiting...
Child is running...
Child is terminating...
SIGCHLD received: Child has terminated.
Parent finished.

```

Program 4 - SIGPIPE - Broken Pipe:

```

GNU nano 8.7.1 signal4.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    int fd[2];
    pipe(fd);
    close(fd[0]); // Close read end
    printf("Writing to pipe...\n");
    write(fd[1], "Hello", 5);
    printf("Write completed.\n");
    return 0;
}

```

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ nano signal4.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ gcc signal4.c -o signal4
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ ./signal4
Writing to pipe...

```

Program 5 - kill() program:

```

GNU nano 8.7.1 signal5.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handler(int sig)
{
    printf("Child received SIGUSR1\n");
}

int main()
{
    pid_t pid = fork();
    if (pid == 0)
    {
        // Child
        signal(SIGUSR1, handler);
        printf("Child waiting...\n");
        while (1)
        {
            sleep(1);
        }
    }
    else
    {
        // Parent
        sleep(2);
        printf("Parent sending signal...\n");
        kill(pid, SIGUSR1);
        sleep(2);
        kill(pid, SIGTERM);
        wait(NULL);
    }
    return 0;
}

```

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ nano signal5.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ gcc signal5.c -o signal5
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ ./signal5
Child waiting...
Parent sending signal...
Child received SIGUSR1

```

Program 6 - SIGABRT — Abort:

```

GNU nano 8.7.1 signal6.c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    printf("Program started\n");
    abort();
    printf("Program ended\n");
    return 0;
}

```

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ nano signal6.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ gcc signal6.c -o signal6
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ ./signal6
Program started
Aborted
./signal6

```

Program 7: SIGSEGV - Invalid Memory Access:

```
GNU nano 8.7.1 signal7.c
#include <stdio.h>
int main()
{
    int *p = NULL;
    printf("Before error\n");
    *p = 10;
    printf("After error\n");
    return 0;
}
```

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ nano signal7.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ gcc signal7.c -o signal7
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Signals$ ./signal7
Before error
Segmentation fault ./signal7
```